

The Guarded Instrument

Sense, trust, decide, act, show, stream, record — the whole track assembled into one honest node. Every guard doing its job at once.

ESP32-010 · ~55 MIN · ADVANCED

1. The one idea

Assemble the whole track into one honest node: **sense** a real value, **trust** it, **decide** under a guard, **act** on a real output, **show** it, **stream** it live, and **record** a replayable trace. The complete guarded instrument — every piece you built, working together.

2. Why it matters

Every course taught one skill; a real instrument is all of them at once. A field node is a sensor, a decision you can trust, an action that's electrically safe, a display, a live view, and an evidence trace — and the interesting part is how they *compose*, and where each guard earns its place. Build it once and the whole track clicks into a single shape.

3. Parts & wiring

Everything from the track, on one board:

- ESP32 + BME280 + SSD1306 OLED on I2C (Courses 3–4)
- A MOSFET-driven output — relay, buzzer, or 12V load — on GPIO25 with its gate network and flyback diode (Course 5)
- WiFi + a WebSocket stream and trace ring (Courses 7–8)

4. Predict (do this before uploading)

- Which guards does this node need at once — input trust, output limit, fail-OFF, non-blocking loop? Name where each lives.
- If WiFi drops, should the node stop sensing and acting, or keep going?
- If the sensor faults, should the output be allowed to fire?

5. Build it — one honest loop

```
// Composed from the track: readTrusted() (C4), the output guard (C1),
// safe MOSFET drive (C5), the OLED (C3), and the WebSocket + ring (C8).
const float ACT_LIMIT = 30.0;          // act above this temperature

void loop() {
  ws.loop();                          // non-blocking (C7/C8) – never delay() the
  network away

  float t;
  Trust s = readTrusted(t);            // C4: present? in range? fresh?
  bool trusted = (s == OK);

  bool act = trusted && (t > ACT_LIMIT); // C1/C4: only act on a trusted reading
  digitalWrite(GATE, act ? HIGH : LOW); // C5: safe MOSFET drive, fails OFF

  showState(t, s, act);                // C3: OLED shows value, trust, action
  publish(t, act ? ACT_LIMIT : t,      // C8: stream the decision ...
          !trusted ? "fault" : act ? "act" : "hold"); // ...and ring-buffer it
}
```

The new idea is **composition**: the action is gated on *trust* (a faulted sensor can't fire the output), the output is driven *safely* (the MOSFET + flyback from Course 5), the loop never blocks (so the stream stays live), and every decision is shown and recorded. Each guard from the track is one clause in this loop.

6. Test against your prediction

Warm the sensor past the limit: the output fires, the OLED shows **ACT**, the dashboard streams it, the trace records it. Now pull the sensor wire — the node fails safe: output **off**, OLED shows **FAULT**, and the fault is logged, *not* acted on. Drop WiFi and the local sensing and acting carry on while the ring buffer keeps the record. Did the faulted sensor refuse to fire the output, as you predicted?

7. What goes wrong (pre-loaded debugging)

- **The output fires on a faulted sensor** → you gated **act** on the value but not on **trusted**. Require both.

- **The stream freezes when the node is busy** → a blocking `delay()` is starving `ws.loop()`. Keep the loop non-blocking.
- **The board resets when the output switches** → the Course 5 power discipline (flyback diode, separate supply) isn't in place on this build.
- **OLED stutters or the loop slows** → heavy I2C redraws every cycle; update the display a few times a second, not every loop.
- **Trace or WiFi eats all the RAM** → unbounded buffers; keep the fixed ring and bound every growing structure.

8. Checkpoint

A complete guarded instrument: it senses, trusts, decides, acts safely, shows, streams, and records — every guard from the track doing its job at once. That is the Monogate node in miniature: an honest decision, made visible and kept. It's a teaching instrument, and its claims stay bounded — but the discipline in it is the real thing. **The ESP32 track is complete.**

9. Push further

- Export the ring buffer as a claim-bounded **evidence packet** — the bench's standard artifact.
- Package it as a field capsule: enclosure, power, and a labelled boundary of what it does and doesn't claim.
- Put two nodes on the network and cross-check their readings — disagreement becomes its own signal.